

Nome: André Machado Silva

Repositório no GitHub:

<https://github.com/andrems1/onboarding-LIPAI>

Código das Videoaulas

```
import matplotlib.pyplot as plt
import numpy as np

x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 6, 3]
plt.plot(x, y, marker='o', color='#1a56db', linewidth=2, label='série A'
)
plt.title('Gráfico de Linha')
plt.xlabel('Eixo X')
plt.ylabel('Eixo Y')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

categorias = ['A', 'B', 'C', 'D']
valores     = [15, 30, 22, 10]
plt.bar(categorias, valores, color='#f97316', edgecolor='white')
plt.title('Gráfico de Barras')
plt.tight_layout()
plt.show()

dados = np.random.randn(500)
plt.hist(dados, bins=25, color='#16a34a', edgecolor='white', alpha=0.85)
plt.title('Histograma')
plt.tight_layout()
plt.show()

x = np.random.rand(100)
y = np.random.rand(100)
cores = np.random.choice(['#1a56db', '#dc2626'], 100)
plt.scatter(x, y, c=cores, alpha=0.7, s=50)
plt.title('Scatter Plot')
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot([1,2,3],[4,5,6])
axes[0].set_title('Subplot 1')
axes[1].bar(['A', 'B', 'C'],[3,7,5])
axes[1].set_title('Subplot 2')
plt.tight_layout()
plt.show()
```

Exercícios

ex01

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('classification_results_trial_0001.csv')
df['acerto'] = df['real_class'] == df['predicted_class']

TP = len(df[(df['real_class']=='malign') & (df['predicted_class']=='malign')])
TN = len(df[(df['real_class']=='benign') & (df['predicted_class']=='benign')])
FP = len(df[(df['real_class']=='benign') & (df['predicted_class']=='malign')])
FN = len(df[(df['real_class']=='malign') & (df['predicted_class']=='benign')])

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

counts_real = df['real_class'].value_counts()
axes[0,0].bar(counts_real.index, counts_real.values,
              color=['#dc2626', '#1a56db'], edgecolor='white', width=0.5)
axes[0,0].set_title('Contagem por Classe Real')

counts_pred = df['predicted_class'].value_counts()
axes[0,1].bar(counts_pred.index, counts_pred.values,
              color=['#dc2626', '#1a56db'], edgecolor='white', width=0.5)
axes[0,1].set_title('Contagem por Classe Predita')

axes[0,2].hist(df['prob_benign'], bins=20, color='#1a56db', edgecolor='white')
axes[0,2].set_title('Histograma - prob_benign')

axes[1,0].hist(df['prob_malign'], bins=20, color='#dc2626', edgecolor='white')
axes[1,0].set_title('Histograma - prob_malign')

acertos = df[df['acerto']]
erros = df[~df['acerto']]
axes[1,1].scatter(acertos['prob_benign'], acertos['prob_malign'],
                  c='#16a34a', alpha=0.65, s=50, label=f'Acerto ({len(acertos)})')
axes[1,1].scatter(erros['prob_benign'], erros['prob_malign'],
                  c='#dc2626', alpha=0.9, s=80, marker='X', label=f'Erro ({len(erros)})')
axes[1,1].set_title('Scatter - Acerto vs Erro')
axes[1,1].legend()

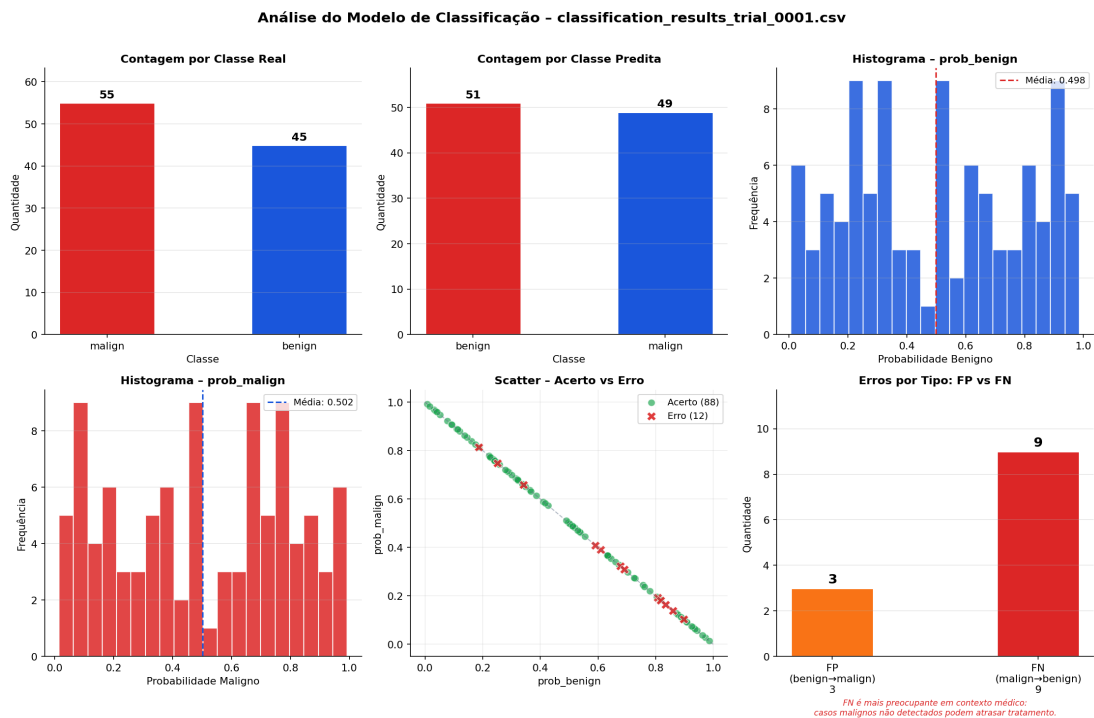
axes[1,2].bar(['FP (benign->malign)', 'FN (malign->benign)'],
              [FP, FN], color=['#f97316', '#dc2626'], edgecolor='white',
              width=0.4)
```

```
axes[1,2].set_title('Erros: FP vs FN')
```

```
plt.tight_layout()
```

```
plt.savefig('ex01_graficos.png', dpi=150, bbox_inches='tight')
```

Gráficos gerados:



O erro mais comum é FN (9 casos), onde o modelo classifica imagens malignas como benignas. Em contexto médico, o FN é mais preocupante pois significa que casos malignos não são detectados, podendo atrasar o diagnóstico e o tratamento do paciente.

ex02

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
df = pd.read_csv('metrics.csv')
```

```
epochs = range(1, len(df) + 1)
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

```
ax1.plot(epochs, df['train_acc'], color='#1a56db', linewidth=2, label='train')
```

```
ax1.plot(epochs, df['val_acc'], color='#f97316', linewidth=2, label='valid')
```

```
ax1.set_title('model accuracy')
```

```
ax1.set_xlabel('epoch')
```

```
ax1.set_ylabel('accuracy')
```

```
ax1.legend()
```

```
ax1.grid(alpha=0.25)
```

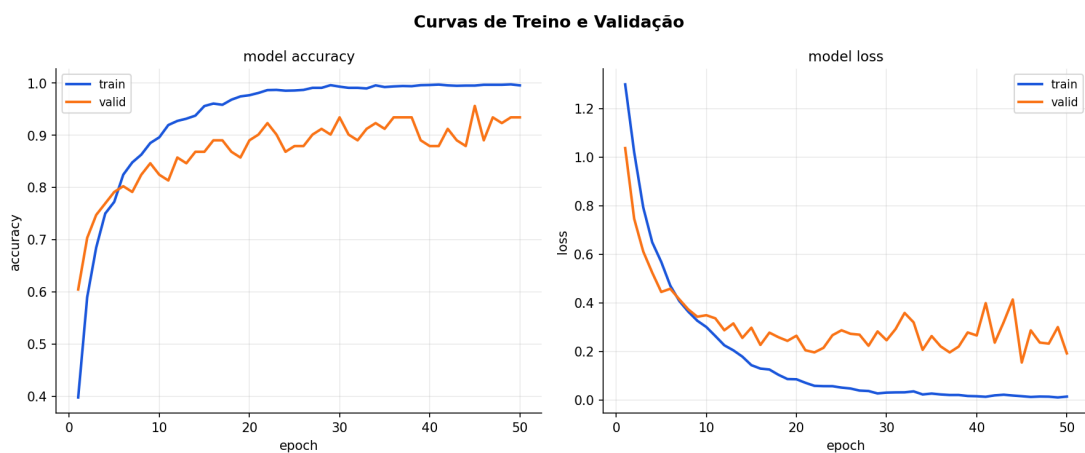
```

ax2.plot(epochs, df['train_loss'], color='#1a56db', linewidth=2, label='
train')
ax2.plot(epochs, df['val_loss'], color='#f97316', linewidth=2, label='
valid')
ax2.set_title('model loss')
ax2.set_xlabel('epoch')
ax2.set_ylabel('loss')
ax2.legend()
ax2.grid(alpha=0.25)

plt.tight_layout()
plt.savefig('ex02_graficos.png', dpi=150, bbox_inches='tight')

```

Gráficos gerados:



A acurácia de treino sobe continuamente enquanto a de validação estabiliza, e a loss de validação começa a subir nas épocas finais — sinais claros de overfitting. O modelo aprende bem nas primeiras épocas mas perde capacidade de generalização.